

생활 맞춤형 식단 및 체중 관리 애플리케이션
청춘 일상 다이어트 총력전

1. 사업 개요

사업명 (프로젝트명): 청춘 일상 다이어트 총력전

- 발주자 (작성자): 오은찬
- 개발 기한: 2026 년 9 월 9 일 ~ 2026 년 9 월 28 일
- 추진 배경 및 목적: 건강 및 체중 관리를 위해서는 매일 섭취한 음식과 체중 변화를 지속적으로 기록하고 확인할 필요가 있다. 그러나 식단과 체중을 각각 따로 기록할 경우 하루 섭취량과 체중 변화의 관계를 확인하기 어렵다. 이에 날짜 별 식단과 체중을 한 프로그램에서 관리하고, 섭취한 음식의 칼로리를 계산하여 사용자가 자신의 식습관과 체중 변화를 쉽게 확인할 수 있는 프로그램을 개발하고자 한다

2. 개발 대상 및 환경 (제약 사항)

- 대상 플랫폼: GUI
- 사용 프로그래밍 언어: Python
- 데이터 저장 방식: JSON
- 기타 개발 제약조건: 데이터베이스 사용 금지(과제 규정 준수), 외부 라이브러리 사용 최소화 (과제의 복잡한 기능 금지" 조건)

3. 상세 요구사항

3.1 기능 요구사항 (Functional Requirements)

요구사항 ID	기능명	상세 내용 및 처리 로직	우선순위
FR-01	기본 기능 1: 식단 기록 및 관리	상세 내용: 날짜, 음식 명, 섭취 칼로리, 식사 시간(아침, 점심, 저녁, 야식)등을 입력하여 식단 기록으로 저장하고 날짜 별 식단 내역 조회 및 등록된 식단 기록의 삭제 기능을 제공한다. 처리로직: 입력한 식단 정보는 날짜별로 저장한다. 사용자가 날짜를	상

		선택하면 해당 날짜에 등록된 식단 정보를 조회할 수 있으며, 선택한 식단 기록을 삭제할 수 있도록 한다.	
FR-02	기본 기능2: 체중 기록 및 변화 조회	상세 내용: 날짜별 체중을 입력하여 저장하고 날짜별 체중 기록을 조회한다. 현재 날짜를 기준으로 최근 7일간의 체중 기록을 조회하여 순서대로 출력하고, 가장 오래된 체중 기록과 현재 체중을 비교하여 증가 또는 감소량을 계산해 출력한다. 처리 로직: 입력한 체중을 날짜별로 저장하고 조회할 수 있도록 한다. 최근 7일간의 체중 기록을 날짜순으로 표시하고, 가장 오래된 체중과 현재 체중을 비교하여 증가 또는 감소량을 계산한다.	상
FR-03	추가 기능 1: 일일 섭취 칼로리 계산	상세 내용: 특정 날짜에 등록된 음식의 섭취 칼로리를 합산하여 하루 총 섭취 칼로리를 계산하고 출력한다. 처리 로직: 선택한 날짜에 등록된 음식들의 섭취 칼로리를 모두 합산하여 일일 총 섭취 칼로리를 계산하고 출력한다.	상
FR-04	추가 기능2: 다이어트 총력전 리포트	상세 내용: 일일 총 섭취 칼로리와 목표 칼로리를 비교하여 당일 목표 달성 여부를 판단하고 성공 또는 실패 결과 문구를 출력한다. 또한 해당 날짜의 식단 기록과 최근 체중 변화 결과를 함께 표시하여 하루 동안의 식단 및 체중 관리 결과를 종합적으로 제공한다. 처리 로직: 해당 날짜의 일일 총 섭취 칼로리와 목표 칼로리를 비교하여 목표 달성 여부를 판단한다. 이후 해당 날짜의 식단 기록, 총 섭취 칼로리, 최근	중

		체중 변화와 목표 달성 결과를 함께 표시한다.	
--	--	------------------------------	--

3.2 데이터 요구사항 (Data Requirements)

•**영속성(Persistence):** 프로그램 종료 시 JSON 파일로 자동 저장하고, 재시작 시 자동 불러오기를 수행한다. records의 날짜별 DailyRecord 데이터는 records.json에 저장하고, 음식 ID별 Food 데이터는 foods.json에 별도로 저장하여 영속성을 유지한다. records에는 날짜별 체중 정보와 식사 시간별 음식 ID 배열을 저장하고, foods에는 음식 ID를 Key로 하여 음식명과 섭취 칼로리 정보를 저장한다. recent_weight_queue는 최근 7일간의 체중 변화를 조회하기 위해 프로그램 실행 중 사용하는 임시 Queue이며 JSON에는 별도로 저장하지 않는다. 목표 칼로리는 전역변수 DAILY_GOAL로 관리한다.

```
{ } 1.json > ...
1  {
2    "2026-01-01": {
3      "weight": 80,
4      "foodlist": {
5        "breakfast": [1, 3],
6        "lunch": [2, 4],
7        "dinner": [5, 6]
8      }
9    }
10 }
```

그림 1) 날짜별 식단 기록 JSON 구조

```
{ } 1.json > ...  
1   {  
2     "1": {  
3       "name": "밥",  
4       "calories": 300  
5     },  
6     "2": {  
7       "name": "계란",  
8       "calories": 150  
9     }  
10  }
```

그림 2) 음식 정보 JSON 구조

- 관리할 데이터 구조:

데이터	설명	예시
날짜	식단 및 체중 기록을 구분하는 날짜	2026-09-05
체중	해당 날짜에 측정한 체중	92.3 kg
식사 시간	음식을 섭취한 시간 구분	아침, 점심, 저녁, 야식
음식 ID	각 음식을 구분하기 위한 식별 번호 1	
음식명	섭취한 음식의 이름	밥
섭취 칼로리	해당 음식의 섭취 칼로리	300 kcal
목표 칼로리	하루 섭취 목표 칼로리	1800 kcal

```

main.py > ...
1  DAILY_GOAL = 1800
2
3  records = {
4      "2026-09-05": DailyRecord(
5          weight=92.3,
6          foodlist={
7              "breakfast": [2],
8              "lunch": [1],
9              "dinner": [],
10             "late_night": []
11         }
12     )
13 }
14
15 foods = {
16     1: Food(name="밥", calories=300),
17     2: Food(name="계란", calories=150)
18 }
19
20 recent_weight_queue = [
21     ("2026-08-30", 93.1),
22     ("2026-08-31", 92.9),
23     ...
24 ]

```

그림 3) 프로그램 내부 데이터 구조

3.3 예외 및 예외 처리 요구사항 (Exception HandLing)

•**EH-01 (빈 값 입력):** 사용자가 음식명, 섭취 칼로리, 체중 등의 값을 입력하지 않은 경우 프로그램이 중단되지 않도록 항목별 기본값을 적용한다. 음식명은 "미입력", 섭취 칼로리는 0, 체중은 None으로 설정한다. 식사 시간을 입력하지 않은 경우 올바른 식사 시간을 다시 입력하도록 한다.

•**EH-02 (잘못된 값 입력):** 섭취 칼로리 또는 체중에 숫자가 아닌 값이나 허용 범위를 벗어난 값이 입력되거나, 정의되지 않은 식사 시간이 입력된 경우 오류 메시지를 출력하고 올바른 값을 다시 입력하도록 한다.

EH-03 (존재하지 않는 기록 조회/삭제): 입력한 날짜 또는 음식 ID에 해당하는 기록이 존재하지 않는 경우 프로그램이 종료되지 않도록 안내 메시지를 출력하고 해당 작업을 중단한다.

3.4 비기능 요구사항 (Non-Functional Requirements)

- GUI 화면 가독성: 식단, 체중, 일일 칼로리, 리포트 등의 항목을 화면 영역별로 구분하여 사용자가 각 정보를 쉽게 확인할 수 있도록 구성한다.
- 안내 및 오류 메시지: 입력 오류, 존재하지 않는 기록 조회·삭제 등의 상황에서 사용자가 원인을 이해할 수 있도록 구체적인 안내 메시지를 출력한다
- 응답 속도: 날짜별 기록 조회, 음식 기록 조회·삭제, 칼로리 계산 등의 기능이 사용자의 입력 후 즉시 결과를 확인할 수 있을 정도로 빠르게 동작하도록 한다.

4. 프로젝트 관리

- 개발자는 기획 및 설계 단계 각각 기획서와 설계서를 제출하여 발주자와 협의하여야 한다.
- 개발자는 구현 중간 단계 필요 시 모듈 테스트 등에 발주자를 참여시켜야 한다.

5. 최종 산출물 (DeLiverables)

개발자는 기한 내에 다음 항목을 완료하여 제출해야 한다.

- **소스 코드 (Source Code):** 동작 가능한 최종 프로그램 코드
- **사용자 가이드 (README.md):** 프로그램 실행 방법 및 기능
 - 1)프로젝트 소개
 - 2)실행 방법
 - 3)주요 기능 설명
 - 4)파일 저장 구조 설명
 - 5)테스트 방법
- **데이터 파일 샘플:** 테스트에 사용된 초기 데이터 파일

6. 검수 및 평가 기준 (Acceptance Criteria)

- [/] FR-01 식단 기록 및 관리: 날짜, 음식명, 섭취 칼로리, 식사 시간을 입력하여 식단을 등록할 수 있으며, 날짜별 식단 조회와 등록된 식단 삭제가 정상적으로 동작하는가?
- [/] FR-02 체중 기록 및 변화 조회: 날짜별 체중을 저장·조회할 수 있으며, 최근 7일간의 체중 기록과 가장 오래된 체중 대비 현재 체중의 증가·감소량이 정상적으로 출력되는가?
- [/] FR-03 일일 섭취 칼로리 계산: 특정 날짜에 등록된 음식의 섭취 칼로리를 합산하여 일일 총 섭취 칼로리가 정상적으로 계산·출력되는가?

- [/] FR-04 다이어트 총력전 리포트: 일일 총 섭취 칼로리와 목표 칼로리를 비교하여 목표 달성 여부를 표시하고, 해당 날짜의 식단 기록과 최근 체중 변화 결과를 함께 제공하는가?
- [/] 데이터 영속성: 프로그램 종료 후 재실행했을 때 기존 식단 및 체중 데이터가 정상적으로 저장·불러오기 되는가?
- [/] EH-01 빈 값 입력: 음식명, 섭취 칼로리, 체중 등의 값이 입력되지 않은 경우 정해진 기본값을 적용하거나 재입력을 요구하며 프로그램이 비정상 종료되지 않는가?
- [/] EH-02 잘못된 값 입력: 칼로리·체중에 숫자가 아닌 값 또는 허용 범위를 벗어난 값이 입력되거나 잘못된 식사 시간이 입력된 경우 오류 메시지를 출력하고 재입력을 요구하는가?
- [/] EH-03 존재하지 않는 기록 조회·삭제: 존재하지 않는 날짜 또는 음식 ID를 조회·삭제하려는 경우 안내 메시지를 출력하고 해당 작업만 중단하는가?
- [/] GUI 화면 가독성: 식단, 체중, 일일 칼로리, 리포트 등의 정보가 구분되어 사용자가 쉽게 확인할 수 있는가?
- [/] 안내 및 오류 메시지: 오류가 발생했을 때 사용자가 원인을 이해할 수 있는 안내 메시지가 제공되는가?
- [/] 응답 속도: 날짜별 기록 조회, 음식 기록 조회·삭제, 칼로리 계산 결과를 사용자가 즉시 확인할 수 있는가?

7.예상 화면

☆ 청춘 일상 다이어트 총력전 ☆

날짜 : 2026-09-05 조회

음식명 : 밥

섭취 칼로리 : 300 kcal

식사 시간 : 점심

식단 등록

식단 삭제

식단 기록

아침 : 계란 / 150 kcal
점심 : 밥 / 300 kcal
저녁 : 없음
야식 : 없음

체중

체중 : 92.3 kg 체중 저장

일일 총 섭취 칼로리

450 / 1800 kcal

최근 체중 변화

7일 전 93.1 kg → 현재 92.3 kg
0.8 kg 감소

결과

목표 달성 성공!